

PROBLEM STATEMENT

AUTONOMOUS PRECISION LANDING ON DYNAMIC PLATFORMS

Track: UAV Dynamics & Autonomous Systems

Overview

Traditional Unmanned Aerial Vehicles (UAVs) rely heavily on static GPS coordinates and fixed landing pads to execute automated touch-downs. However, the next frontier of autonomous robotics demands high-speed dynamic interaction. Whether it is a search-and-rescue quadcopter landing on the pitching deck of a marine vessel, a delivery drone docking with a moving freight truck, or an exploration vehicle synchronizing with a planetary rover, target platforms are rarely stationary.

Landing on a moving target requires a fundamental shift from static waypoint navigation to dynamic, real-time tracking. It demands a seamless integration of computer vision for target acquisition, predictive state estimation to calculate relative momentum, and highly responsive control loops to counteract drift and prevent catastrophic collisions upon touchdown.

Challenge

Your mission is to develop a real-time, autonomous flight controller for a quadcopter operating in a simulated 3D physics environment. The quadcopter must visually locate a ground rover moving in an unpredictable, continuous trajectory, synchronize its velocity to maintain a stable hover directly above the rover, and execute a mathematically controlled descent.

Upon making physical contact with the rover's landing pad, the drone must instantly recognize the touchdown and cut all motor thrust to prevent the physics engine from flipping or dragging the aircraft.

Scope and Technology Stack

- **Physics Simulation:** The environment is built in PyBullet. It features idealized rigid-body dynamics, surface friction, and gravity. Aerodynamic drag and motor spool-up lag are omitted to isolate your control logic.
- **Computer Vision:** You will use OpenCV (cv2, specifically the cv2.aruco module) to process a simulated downward-facing RGB camera feed.
- **Mathematics & State:** You are expected to use NumPy and standard Python math libraries for matrix operations, quaternions, and filtering.
- **Environment Constraints:** You will be provided with a `local_arena.py` testing script that manages the rover kinematics and evaluation metrics. You will only write and submit the `modular_controller.py` file.

Core Tasks

- **Visual Acquisition & Processing:** Ingest the raw pixel data from the simulated downward camera. Use OpenCV to detect the unique ArUco fiducial marker mapped to the rover, extracting its pixel coordinates and orientation relative to the drone's optical center.
- **State Estimation:** Transform 2D pixel errors and raw PyBullet telemetry (quaternions, absolute coordinates, and velocities) into actionable 3D relative tracking metrics.
- **Kinematic Control Architecture:** Design and tune a multi-variable control system (e.g., cascaded PID loops). Your controller must ingest the positional error and output the raw physical thrust forces (in Newtons) required for all four independent rotors to control pitch, roll, yaw, and altitude.
- **Landing State Machine:** Implement a logical sequence that transitions the quadcopter through distinct phases:

Target Search → Synchronized Tracking → Controlled Descent → Emergency Motor Cutoff.

- **Physical Threshold Compliance:** Your descent must be mathematically smooth. The evaluation script will automatically fail any landing that exceeds a Z-axis impact velocity of -0.65 m/s or lands with a chassis tilt exceeding 30 degrees.

Deliverables

- **Report (PDF):** Must include your mathematical approach, control system block diagrams, vision pipeline logic, and planned state machine. Should clearly explain your logic, PID tuning process and how you handled the edge cases.
- **README.md:** Clear setup steps, required library dependencies (e.g., specific OpenCV versions), and a brief overview of your solution.
- **Demo Video Link:** A screen recording of your quadcopter successfully tracking and landing on the rover in the PyBullet arena.
- **Zip file:** Containing your final controller.py code and any custom auxiliary Python modules you developed, README.md, Report, Demo Video.

Evaluation Criteria

- **Vision & Estimation (20%):** ArUco marker tracking and 2D-to-3D spatial mapping accuracy.
- **Control Stability (25%):** Controller logic, thrust mixing, and smooth flight tracking.
- **Landing Execution (35%):** Touchdown precision, safe impact limits, and successful motor cutoff.
- **Final Report (20%):** Mathematical justifications, block diagrams, and code documentation.